56. Явное преобразование (приведение) типов данных

Из предыдущего урока мы уже знаем, что компилятор в определенных случаях выполняет **неявное преобразование типов данных**. На этом уроке мы рассмотрим использование явной конвертации типов данных.

Оглавление:

1. Зачем использовать явную конвертацию данных?
2. Операторы явного преобразования типов данных
3. Конвертация C-style
4. Оператор static_cast
5. Использование операторов явного преобразования в неявном преобразовании
6. Заключение
7. Тест

Зачем использовать явную конвертацию данных?

Когда вы хотите изменить один тип данных на другой, более крупный (по размеру/диапазону), то неявное преобразование является хорошим вариантом.

Но многие начинающие программисты часто пытаются сделать что-то вроде следующего: `float x = i1 / 3;`. Однако, поскольку `i1` и `3` являются целыми числами, никакого числового расширения не происходит. Выполняется целочисленное деление `i1 / 3`, результатом которого будет значение `3`, которое затем неявно преобразуется в `3.0` и присвоится переменной `x`!

В случае, когда вы используете **литералы** (такие как `i1` или `3`), замена одного или обоих целочисленных литералов значением **типа с плавающей точкой** (`i1.0` или `3.0`) приведет к конвертации обоих операндов в значения типа с плавающей точкой и выполнится деление типа с плавающей точкой.

Но что будет, если использовать переменные? Например:

```
int i1 = 11;
int i2 = 3;
float x = i1 / i2;
```

Значением переменной `x` будет `3`. Как сообщить компилятору, что мы хотим использовать деление типа с плавающей точкой вместо целочисленного деления? Правильно! Использовать один из операторов явного преобразования типов данных, чтобы указать компилятору выполнить явное преобразование.

Операторы явного преобразования типов данных

В языке C++ есть 5 видов **операций явного преобразования типов**:

- **конвертация C-style**;
- **применение оператора static_cast**;
- **применение оператора const_cast**;
- **применение оператора dynamic_cast**;
- **применение оператора reinterpret_cast**.

На этом уроке мы рассмотрим конвертацию C-style и оператор static_cast. Оператор dynamic_cast мы будем рассматривать, когда дойдем до указателей и наследования. Применения операторов const_cast и reinterpret_cast следует избегать, так как они полезны только в редких случаях и могут создать немало проблем, если их использовать неправильно.

Правило: Избегайте использования операторов const_cast и reinterpret_cast, если у вас нет на это веских причин.

Конвертация C-style

В программировании на языке Си явное преобразование типов данных выполняется с помощью оператора `()`. Внутри круглых скобок мы пишем тип, в который нужно конвертировать. Этот способ конвертации типов называется **конвертацией C-style**. Например:

```
int i1 = 11;
int i2 = 3;
float x = (float)i1 / i2;
```

В программе, приведенной выше, мы используем круглые скобки, чтобы сообщить компилятору о необходимости преобразования переменной `i1` (типа int) в тип float. Поскольку переменная `i1` станет типа float, то `i2` также затем автоматически преобразуется в тип float, и выполнится деление типа с плавающей точкой!

Язык C++ также позволяет использовать этот оператор следующим образом:

```
int i1 = 11;
int i2 = 3;
float x = float(i1) / i2;
```

Конвертация C-style не проверяется компилятором во время компиляции, поэтому она может быть неправильно использована, например, при конвертации типов const или изменении типов данных, без учета их диапазонов (что может привести к **переполнению**).

Следовательно, конвертацию C-style лучше не использовать.

Правило: Не используйте конвертацию C-style.

Оператор static_cast

В языке C++ есть еще один оператор явного преобразования типов данных — **оператор static_cast**. Ранее, на уроке о **символьном типе данных char**, мы уже использовали оператор static_cast для конвертации переменной типа char в тип int, выводя вместо символа целое число:

```
char c = 97;
std::cout << static_cast<int>(c) << std::endl; // в результате выведется 97, а не 'a'
```

Оператор static_cast лучше всего использовать для конвертации одного фундаментального типа данных в другой:

```
int i1 = 11;
int i2 = 3;
float x = static_cast<float>(i1) / i2;
```

Основным преимуществом оператора static_cast является проверка его выполнения компилятором во время компиляции, что усложняет возможность возникновения непреднамеренных проблем.

Использование операторов явного преобразования в неявном преобразовании

Если вы будете выполнять небезопасные неявные преобразования типов данных, то компилятор будет жаловаться. Например:

```
int i = 49;
char ch = i; // неявное преобразование
```

Конвертация переменной типа int (4 байта) в тип char (1 байт) потенциально опасна — компилятор выдаст предупреждение. Чтобы сообщить ему, что вы намеренно делаете что-то, что потенциально опасно (но хотите сделать это в любом случае), используйте оператор static_cast:

```
int i = 49;
char ch = static_cast<char>(i);
```

В следующем случае компилятор будет жаловаться, что конвертация из типа double в тип int может привести к потере данных:

```
int i = 90;
i = i / 3.6;
```

Чтобы сообщить компилятору, что мы сознательно хотим сделать это:

```
int i = 90;
i = static_cast<int>(i / 3.6);
```

Заключение

Преобразования типов данных следует избегать, если это вообще возможно, поскольку всякий раз, когда выполняется подобное изменение, есть вероятность возникновения непредвиденных проблем. Но очень часто случаются ситуации, когда этого не избежать. Поэтому в таких случаях лучше использовать оператор static_cast вместо конвертации C-style.

Тест

В чём разница между явным и неявным преобразованием типов данных?

Ответ

Неявное преобразование происходит, когда компилятор ожидает значение одного типа, но получает значение другого типа.

Явное преобразование происходит, когда программист использует оператор явного преобразования для конвертации значения из одного типа данных в другой.